

Teach It to Judge

**Don't ask the model to score.
Ask it to read.**

Your database is a pile of real listings with no opinion attached to any of them. Today it gets judgement — reproducible, explainable, testable judgement, which is not what you get by asking an LLM for a number out of a hundred.

Where we left off

- 1 A real **Fetcher** pulling genuine listings off a live job board.
- 2 A **source layer** where every board is a plug-in, and one HTTP helper with a timeout.
- 3 **50+ real listings** in your own database — and no opinion about a single one of them.

RIGHT NOW YOUR DATABASE IS A PILE

Fifty rows, all equally important, sorted by nothing. Same problem you started the month with — **just in SQLite instead of a browser tab.**

No real listings? **use_mock_fetcher: true** — everything today works identically on the 12 mock rows. That's why we kept the flag.

A number is not a measurement.

The obvious build: hand the listing and your profile to a model and ask it to score the fit out of a hundred. It will answer instantly, confidently, every time — and explain itself convincingly.

WHY THAT NUMBER IS NEARLY WORTHLESS

A language model predicts plausible text. **“Eighty-seven” is plausible text.** Nothing in that process counted anything.

Four ways an LLM scorer lies to you

1

Inflation. Ask out of 100, get 85–92 on nearly everything.

Models are trained to be agreeable and agreeable looks like a high score. A ranking where everything ranks first is not a ranking.

2

Non-determinism. Same listing twice → 72, then 88. Which is true?

Neither — there was never a true number to find. And you cannot debug a system whose output changes when nothing changed.

3

The unparseable response. You asked for JSON. You got a markdown fence and a greeting.

`json.loads` throws at 6 AM on listing 41 of 50 — and 42 through 50 never get scored at all.

4

Cost and time. 50 listings is 50 calls. Every morning. Forever.

Re-score everything daily and by week 4 you're making thousands of calls to produce answers you already had.

Don't ask it to score. Ask it to read.

THE MODEL DOES

- 1 Reads the job description
- 2 Returns **facts**: skills, seniority, remote
- 3 Never sees your weights

YOUR CODE DOES

- 1 Compares facts to your profile
- 2 Computes four **weighted components**
- 3 Produces the score and the reason

Reading comprehension is what it's **for**. Set intersection and multiplication is five lines of Python that gives the same answer every time, free, in a microsecond. **Split the job.**

What you built today

- 1 **One door to the model** — with a retry that tells the model what it got wrong.
 - 2 An **extractor** that reads job descriptions, never scores them, and is cached on the text hash.
 - 3 A **deterministic scoring function** — your weights, your config, and unit tests.
 - 4 Reasons **generated from arithmetic**, not written by a model — so they're always true.
 - 5 A **distribution logged every run** — which is next week's tripwire.
- Total AI in the file that decides your rankings: **none**. That's the point.

Every listing scored, with a real spread.

89

...and 34.

Not 89 and 84. If your top and bottom are within 15 points, something is wrong — start with your config skills list.

Also due Sunday

- 1 `python -m edgedash.llm --check` passes
- 2 Second cycle scores **zero** — idempotence
- 3 `tests/test_scoring.py` passing, 6+ cases
- 4 Reason string on every scored row
- 5 Distribution logged to `cycle_log`
- 6 `.env` still gitignored, key never committed

Score it twice. Get the same number.

Clear one listing's score, re-score it, and put both numbers on screen. **A naive “ask the model for a score” build cannot do this.**

THE PROOF

```
$ python -m edgedash.rescore --id 41ac9e
```

```
cleared 1 score · extraction cache untouched
```

```
$ python run_cycle.py
```

```
scorer scored 1 · range 89-89 · 0 failed
```

```
before: 89    after: 89    ← same input, same output
```

Say **why** it comes out the same, in one sentence. If you can explain that, you've got the whole session.

Bring something broken.

Live debugging session — **[day, date · time]**. You bring a build that doesn't work, I debug it on screen.

THE INSTINCT TO RESIST

Everyone wants to show up only if their code works. **Come with the broken thing — that's the session.**

You'll get more from watching one real bug diagnosed properly than from three more videos of mine. And the person whose bug we look at is never the only person in the room who has it.

No prep needed. **Bring the error message and the file.** That's enough.

Two things while it's fresh

1

Read your own **bottom five scores**.

Sort ascending and look at the worst matches. Twice now I've found my CONFIG was wrong, not my code — a skill I actually have that I'd forgotten to list. Your scorer is an honest mirror of your config.

2

Post your top five **with the reason strings**.

That column is the most interesting thing you've built this month and it explains itself to a non-technical reader. LinkedIn, #MyEdge, one sentence on how the score is computed.

WORTH SAYING IN THE POST

“The model doesn't produce the number — it reads the listing, my code does the arithmetic.” Most people will assume the opposite.

What is it costing you to not know Kubernetes?



Every listing already knows what it's missing — that's the “gap:” part of the reason string you generated today, sitting on every row. Thursday we read across all of them and count.

Stable is not the same as right.

I showed you that 89 comes back as 89. I never showed you that 89 is the correct answer for that listing — and I can't, because I don't know what the correct answer is either.

WHAT YOU ACTUALLY HAVE

A system that is **wrong in a consistent, inspectable way**. Which is a real step up from wrong at random — and is not the finish line.

Extraction can still miss a skill or misread a band — quietly, confidently, with a dashboard that looks fine. **That's week 3.**

Your system has an opinion now.

Reproducible, explainable, tunable, tested — and cheap enough to run every morning for the rest of the month without thinking about it.

Next week you build the agent whose entire job is to distrust the rest of your system. But you couldn't have built it against a scorer that answered differently every time.

Every listing scored, with a spread. Score one thing twice. See you Thursday.